Zero Trust Driven Application Trust Establishment Framework in SDN

Project Report

1. Problem Statement

In traditional Software-Defined Networking (SDN), the Northbound API allows network applications to communicate with the controller. However, there is no built-in authentication or authorization mechanism. Once an application is installed, it is implicitly trusted by the controller. This creates several security risks:

- A compromised application can inject fake flow rules and modify network state.
- A fake application can impersonate a legitimate service.
- Replay attacks can reuse captured tokens to gain unauthorized access.
- An application can perform operations beyond its intended function (privilege escalation).

Existing solutions (FortNox, Rosemary, PermOF) provide either authentication or behavioral containment, but none combine continuous verification, dynamic trust scoring, and policy enforcement in a single framework. This project fills that gap by implementing Zero Trust principles (NIST SP 800-207) in the SDN environment.

2. What Was Built

A Zero Trust verification layer between network applications and the SDN controller. Every request from every application is verified before it can modify the network.

2.1 System Components

Component	File	Purpose
SDN Controller	`ryu_controller.py`	Ryu-based controller with OpenFlow 1.3 support, L2 learning switch, and Northbound REST API (port 8080). All application requests go through the Zero Trust pipeline.
Trust Verification Engine	`trust_verification.py`	7-layer verification pipeline. Each request is checked across Token, API Key, Device Identity, RBAC, ABAC, Behavioral Analysis, and Trust Scoring.
Policy Engine	`policy_engine.py`	Evaluates policies after verification and decides allow/deny.
Policy Enforcement Point	`policy_enforcement_point.py`	Fail-closed guard. If verification fails, request is denied immediately. If allowed, OpenFlow flow rules are installed on the switch.
Trust Scoring	`trust_scoring.py`	Tracks long-term reputation. Score increases on successful requests (+0.1), decreases on failures (-0.3 for security failures, -0.1 for privilege violations), and decays over time (-0.01/minute).
Baseline Controller	`baseline_controller.py`	Same REST interface but without Zero Trust verification. Used as a control group for comparison.

2.2 Application Layer

- **Legitimate apps:** `http_app_client.py` and `legitimate_app.py` with roles (monitor, admin).
- **Attack simulations (5 types):** `malicious_apps.py` - no credentials, fake device identity, replay attack, unauthorized REST calls, flood attack.

2.3 Trust Model

Trust is calculated as a composite score:

```
Composite Trust = 0.7 × Session Trust + 0.3 × Reputation
```

Session Trust comes from the 7-layer verification of the current request (weighted sum: token 0.20, API key 0.15, device 0.15, RBAC 0.20, ABAC 0.15, behavioral 0.15). A score above 0.6 allows the request.

Reputation is the long-term history maintained by the scoring module. It increases on success and decreases on failure, with time decay.

3. How It Was Done (Implementation Steps)

1. **Framework development:** Built the 7-layer verification pipeline, trust engine, policy enforcement point, and REST API on top of Ryu SDN controller.
2. **Bug identification and fixes (5 critical issues found during testing):**
    - **PEP never checked the "allowed" field** in the trust verdict. This meant behavioral attacks were never actually blocked. Fixed by adding fail-closed early-deny logic in `PEP.enforce()`.
    - **Trust verdict dropped trust_score and reason** from the response. All API responses showed trust=0.0. Fixed by adding full attribution to the enforcement result.
    - **Uniform -0.3 penalty for all failures** caused legitimate apps (like monitor) to get locked out after RBAC boundary probes. Fixed by introducing tiered penalties: security failures (auth/identity/behavioral) get -0.3, while privilege violations (RBAC/ABAC) get -0.1.
    - **Flood threshold was too loose** (10 requests in 5 seconds). Tightened to 6 requests in 3 seconds with progressive burst penalty: trust drops to 0.00 as flooding continues.
    - **Client token timestamp bug** - the client sent a fresh timestamp with the registration token, causing verification failures across second boundaries. Fixed by using the token issue timestamp.
3. **Evaluation setup:** Ubuntu 22.04 VM with VirtualBox, Ryu 4.34, Mininet, OpenFlow 1.3. One OVS switch with 4 hosts (monitor, admin, attacker, user).
4. **Benchmark scripts:** Created `bench_phase_b.py` (detection, latency, trust traces) and `bench_scalability.py` (4/8/16 hosts). Each test runs 50 times for statistical reliability.
5. **Paper writing:** IEEE-format manuscript (`paper.tex`) with system design, threat model, algorithm, results tables, and figures.
6. **GitHub:** All code, paper, and results pushed to a private repository.

4. Results

4.1 Attack Detection (50 runs per class)

Attack Class	Detection Rate	How it was blocked
No credentials	1.000	Token verification failed at Step 1
Fake device	1.000	Device identity check failed at Step 3
Replay attack	1.000	Token signature/timestamp mismatch at Step 1
Unauthorized REST	1.000	RBAC check failed at Step 4
Flood attack	1.000	Behavioral anomaly detected at Step 6 (request_block_rate: 0.600)

4.2 False Positive Rate (legitimate applications)

App	False Positive Rate	Details
legitimate_monitor	0.000 (0 out of 50)	Never incorrectly blocked
legitimate_admin	0.000 (0 out of 50)	Never incorrectly blocked

4.3 Latency Overhead (vs Baseline)

Metric	Baseline	Zero Trust	Overhead
Sequential avg	4.038 ms	5.520 ms	+1.48 ms (+37%)
Sequential p95	7.352 ms	9.078 ms	+1.73 ms
Concurrent avg (10 workers)	25.158 ms	50.181 ms	~2x (due to eventlet GIL)

4.4 Scalability

Hosts	Ping Loss	Sequential Avg	Sequential p95
4	0.0%	3.612 ms	5.204 ms
8	0.0%	3.804 ms	4.869 ms
16	0.0%	3.660 ms	4.905 ms

Latency remains flat as the network scales, confirming that verification cost is per-request and independent of topology size.

4.5 Trust Evolution

- **Admin (legitimate):** Trust starts at 0.88, increases to 1.00 with successful requests.
- **Monitor (legitimate with RBAC probes):** Trust dips to 0.3-0.45 when write-probes are denied, then self-heals as legitimate reads succeed.
- **Compromised app (flood):** Trust drops from 0.88 to 0.00 with progressive lockout.

5. How to Run

Dependencies

```
sudo bash setup/install_dependencies.sh
```

Start Controller

```
sudo ryu-manager controller/ryu_controller.py
```

Start Network

```
sudo python3 topology/network_topology.py
```

Run Applications (from Mininet CLI)

```
h1 python3 ../apps/http_app_client.py --app monitor
h2 python3 ../apps/http_app_client.py --app admin
h3 python3 ../apps/http_app_client.py --attack no_credentials
h3 python3 ../apps/http_app_client.py --attack fake
h3 python3 ../apps/http_app_client.py --attack replay
h3 python3 ../apps/http_app_client.py --attack unauthorized
h3 python3 ../apps/http_app_client.py --attack flood
```

Check Results

`curl http://127.0.0.1:8080/zt/stats` — total requests, allowed, denied, block rate

`curl http://127.0.0.1:8080/zt/trust/legitimate_admin` — current trust score of an app

Run Benchmarks

`python3 tools/bench_phase_b.py` — detection + latency + trust traces

`python3 tools/bench_scalability.py` — 4/8/16 hosts scalability test

6. Limitations

- Mininet emulation (not production SDN hardware).
- Single OVS switch topology.
- Eventlet GIL causes concurrent latency to double.
- Secret keys are hardcoded in the code (production would need HSMs or key vaults).
- Time-based trust decay is linear (could be exponential in production).